FLOLAC 2026

Logic

IV. The Curry–Howard correspondence

柯向上

中央研究院資訊科學研究所

https://josh-hs-ko.github.io

Annotated derivation

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\mathtt{A}, \mathtt{A} \to \mathtt{B} \vdash \mathtt{A} \to \mathtt{B}}\,(\mathsf{A}) \quad \dfrac{}{\mathtt{A}, \mathtt{A} \to \mathtt{B} \vdash \mathtt{A}}\,(\mathsf{A})}{\mathtt{A}, \mathtt{A} \to \mathtt{B} \vdash \mathtt{B}}\,(\to\mathsf{E})}{\mathtt{A} \vdash (\mathtt{A} \to \mathtt{B}) \to \mathtt{B}}\,(\to\mathsf{I})}{\vdash \mathtt{A} \to (\mathtt{A} \to \mathtt{B}) \to \mathtt{B}}\,(\to\mathsf{I})$$

Annotated derivation

$$\dfrac{\dfrac{\dfrac{}{\mathtt{x} : \mathtt{A}, \mathtt{y} : \mathtt{A} \to \mathtt{B} \vdash \mathtt{A} \to \mathtt{B}}\,(\mathsf{A}) \qquad \dfrac{}{\mathtt{x} : \mathtt{A}, \mathtt{y} : \mathtt{A} \to \mathtt{B} \vdash \mathtt{A}}\,(\mathsf{A})}{\mathtt{x} : \mathtt{A}, \mathtt{y} : \mathtt{A} \to \mathtt{B} \vdash \mathtt{B}}\,(\to\mathsf{E})}{\mathtt{x} : \mathtt{A} \vdash (\mathtt{A} \to \mathtt{B}) \to \mathtt{B}}\,(\to\mathsf{I})}{\vdash \mathtt{A} \to (\mathtt{A} \to \mathtt{B}) \to \mathtt{B}}\,(\to\mathsf{I})$$

- Label assumptions with (distinct) names.

Annotated derivation

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\mathtt{x}:\mathtt{A},\mathtt{y}:\mathtt{A}\to\mathtt{B}\vdash \mathtt{y}:\mathtt{A}\to\mathtt{B}}\,(\mathsf{A}) \quad \dfrac{}{\mathtt{x}:\mathtt{A},\mathtt{y}:\mathtt{A}\to\mathtt{B}\vdash \mathtt{x}:\mathtt{A}}\,(\mathsf{A})}{\mathtt{x}:\mathtt{A},\mathtt{y}:\mathtt{A}\to\mathtt{B}\vdash \mathtt{B}}\,(\to\mathsf{E})}{\mathtt{x}:\mathtt{A}\vdash(\mathtt{A}\to\mathtt{B})\to\mathtt{B}}\,(\to\mathsf{I})}{\vdash \mathtt{A}\to(\mathtt{A}\to\mathtt{B})\to\mathtt{B}}\,(\to\mathsf{I})$$

- Label assumptions with (distinct) names.
- Represent (A) by the name of the assumption used.

Annotated derivation

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\mathtt{x} : \mathtt{A}, \mathtt{y} : \mathtt{A} \to \mathtt{B} \vdash \mathtt{y} : \mathtt{A} \to \mathtt{B}}\,(\mathsf{A}) \quad \dfrac{}{\mathtt{x} : \mathtt{A}, \mathtt{y} : \mathtt{A} \to \mathtt{B} \vdash \mathtt{x} : \mathtt{A}}\,(\mathsf{A})}{\mathtt{x} : \mathtt{A}, \mathtt{y} : \mathtt{A} \to \mathtt{B} \vdash \mathtt{y}\ \mathtt{x} : \mathtt{B}}\,(\to\mathsf{E})}{\mathtt{x} : \mathtt{A} \vdash (\mathtt{A} \to \mathtt{B}) \to \mathtt{B}}\,(\to\mathsf{I})}{\vdash \mathtt{A} \to (\mathtt{A} \to \mathtt{B}) \to \mathtt{B}}\,(\to\mathsf{I})$$

- Label assumptions with (distinct) names.
- Represent (A) by the name of the assumption used.
- Represent ($\to$E) by juxtaposing the representations of its two sub-derivations.

Annotated derivation

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\mathtt{x} : \mathtt{A}, \mathtt{y} : \mathtt{A} \to \mathtt{B} \vdash \mathtt{y} : \mathtt{A} \to \mathtt{B}}\,(\mathsf{A}) \quad \dfrac{}{\mathtt{x} : \mathtt{A}, \mathtt{y} : \mathtt{A} \to \mathtt{B} \vdash \mathtt{x} : \mathtt{A}}\,(\mathsf{A})}{\mathtt{x} : \mathtt{A}, \mathtt{y} : \mathtt{A} \to \mathtt{B} \vdash \mathtt{y}\ \mathtt{x} : \mathtt{B}}\,(\to\mathsf{E})}{\mathtt{x} : \mathtt{A} \vdash \lambda\,\mathtt{y}.\ \mathtt{y}\ \mathtt{x} : (\mathtt{A} \to \mathtt{B}) \to \mathtt{B}}\,(\to\mathsf{I})}{\vdash \lambda\,\mathtt{x}.\ \lambda\,\mathtt{y}.\ \mathtt{y}\ \mathtt{x} : \mathtt{A} \to (\mathtt{A} \to \mathtt{B}) \to \mathtt{B}}\,(\to\mathsf{I})$$

- Label assumptions with (distinct) names.
- Represent (A) by the name of the assumption used.
- Represent ($\to$E) by juxtaposing the representations of its two sub-derivations.
- Represent ($\to$I) by prefixing $\lambda\, v.$ to the representation of its sub-derivation, where v is the name of the new assumption.

Annotated derivation

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\mathtt{x}:\mathtt{A},\mathtt{y}:\mathtt{A}\to\mathtt{B}\vdash \mathtt{y}:\mathtt{A}\to\mathtt{B}}\,(\mathsf{var}) \quad \dfrac{}{\mathtt{x}:\mathtt{A},\mathtt{y}:\mathtt{A}\to\mathtt{B}\vdash \mathtt{x}:\mathtt{A}}\,(\mathsf{var})}{\mathtt{x}:\mathtt{A},\mathtt{y}:\mathtt{A}\to\mathtt{B}\vdash \mathtt{y}\ \mathtt{x}:\mathtt{B}}\,(\mathsf{app})}{\mathtt{x}:\mathtt{A}\vdash \lambda\,\mathtt{y}.\ \mathtt{y}\ \mathtt{x}:(\mathtt{A}\to\mathtt{B})\to\mathtt{B}}\,(\mathsf{abs})}{\vdash \lambda\,\mathtt{x}.\ \lambda\,\mathtt{y}.\ \mathtt{y}\ \mathtt{x}:\mathtt{A}\to(\mathtt{A}\to\mathtt{B})\to\mathtt{B}}\,(\mathsf{abs})$$

- Label assumptions with (distinct) names.
- Represent (A) by the name of the assumption used.
- Represent ($\to$E) by juxtaposing the representations of its two sub-derivations.
- Represent ($\to$I) by prefixing $\lambda\, v.$ to the representation of its sub-derivation, where v is the name of the new assumption.

This is a typing derivation for the λ-term $\lambda\,\mathtt{x}.\ \lambda\,\mathtt{y}.\ \mathtt{y}\ \mathtt{x}$!

Simply typed λ-calculus (à la Curry)

Let the set of *types* be the *implicational fragment* of $\textsc{Prop}$, i.e., the subset of the propositional language generated by variables and implication only.

A λ-term t is said to *have type τ under context Γ* if, using the following rules, there is a (finished) typing derivation whose conclusion is $\Gamma \vdash t : \tau$. In this case we simply write $\Gamma \vdash t : \tau$.

$$\frac{}{\Gamma \vdash v : \tau}\ (\text{var}) \quad \text{if} \quad (v : \tau) \in \Gamma$$

$$\frac{\Gamma, v : \sigma \vdash t : \tau}{\Gamma \vdash \lambda v.\ t\ :\ \sigma \to \tau}\ (\text{abs}) \qquad \frac{\Gamma \vdash t\ :\ \sigma \to \tau \qquad \Gamma \vdash s : \sigma}{\Gamma \vdash t\ s\ :\ \tau}\ (\text{app})$$

Exercise. Derive $\vdash (\mathtt{A} \to (\mathtt{B} \to \mathtt{C})) \to (\mathtt{B} \to (\mathtt{A} \to \mathtt{C}))$ in NJ and convert the derivation to a λ-term.

The Curry–Howard correspondence

Deduction systems and programming calculi can be put in correspondence — a corresponding pair of a deduction system and a programming calculus can be regarded as logical and computational interpretations of essentially the same set of syntactic objects.

Slogan: *propositions are types; proofs are programs.*

Natural deduction for full propositional logic corresponds to simply typed λ-calculus with extensions: conjunction correspond to pair types, disjunction to disjoint sum types, and falsity to an empty type.

Question. Are unannotated derivations and λ-terms in one-to-one correspondence?

Disjoint sums

Disjunctions correspond to disjoint sums/unions (EITHER types in Haskell): the introduction rules give types to the injection constructors,

$$\frac{\Gamma \vdash s : \sigma}{\Gamma \vdash \texttt{left}\ s : \sigma \vee \tau}\ (\vee\mathsf{IL}) \qquad \frac{\Gamma \vdash t : \tau}{\Gamma \vdash \texttt{right}\ t : \sigma \vee \tau}\ (\vee\mathsf{IR})$$

and the elimination rule gives type to a pattern-matching operator:

$$\frac{\Gamma \vdash c : \sigma \vee \tau \qquad \Gamma, u : \sigma \vdash s : \vartheta \qquad \Gamma, v : \tau \vdash t : \vartheta}{\Gamma \vdash \texttt{case}\ c \begin{bmatrix} \texttt{left} & u \mapsto s \\ \texttt{right} & v \mapsto t \end{bmatrix} : \vartheta}\ (\vee\mathsf{E})$$

The language of λ-calculus is extended with terms of the forms $\texttt{left}\ s$, $\texttt{right}\ t$, and $\texttt{case}\ c \begin{bmatrix} \texttt{left} & u \mapsto s \\ \texttt{right} & v \mapsto t \end{bmatrix}$.

Exercise. What is the λ-term corresponding to your derivation of $\vdash (\mathtt{A} \vee \mathtt{B}) \to (\mathtt{B} \vee \mathtt{A})$?

β-reduction in λ-calculus

In pure λ-calculus we have β-reduction for rewriting β-redexes.

$$(\lambda\, v.\ s)\ t\ \ \rightarrow_\beta\ \ s\,[t/v]$$

Note that this is how an elimination form (application) cancels out an introduction form (λ-abstraction) for the same connective (Gentzen's inversion principle).

For the extended λ-calculus, we should specify how to reduce additional forms of β-redex that involve the introduction and elimination forms of the other connectives. For example:

$$\texttt{case}\ (\texttt{left}\ \ p)\ \begin{bmatrix}\texttt{left}\ \ u \mapsto s\\ \texttt{right}\ v \mapsto t\end{bmatrix}\ \ \rightarrow_\beta\ \ s\,[p/u]$$

$$\texttt{case}\ (\texttt{right}\ q)\ \begin{bmatrix}\texttt{left}\ \ u \mapsto s\\ \texttt{right}\ v \mapsto t\end{bmatrix}\ \ \rightarrow_\beta\ \ t\,[q/v]$$

Proof normalisation

Redexes in λ-terms correspond to *detours* in derivations, and evaluation of λ-terms corresponds to *proof normalisation*.

$$\dfrac{\dfrac{\dfrac{\dfrac{}{\mathtt{B} \to \mathtt{C} \to \mathtt{B}, \mathtt{A} \vdash \mathtt{B} \to \mathtt{C} \to \mathtt{B}}\,(\mathsf{A})}{\mathtt{B} \to \mathtt{C} \to \mathtt{B} \vdash \mathtt{A} \to \mathtt{B} \to \mathtt{C} \to \mathtt{B}}\,(\to\mathsf{I})}{\vdash (\mathtt{B} \to \mathtt{C} \to \mathtt{B}) \to \mathtt{A} \to \mathtt{B} \to \mathtt{C} \to \mathtt{B}}\,(\to\mathsf{I}) \qquad \dfrac{\dfrac{\dfrac{}{\mathtt{B}, \mathtt{C} \vdash \mathtt{B}}\,(\mathsf{A})}{\mathtt{B} \vdash \mathtt{C} \to \mathtt{B}}\,(\to\mathsf{I})}{\vdash \mathtt{B} \to \mathtt{C} \to \mathtt{B}}\,(\to\mathsf{I})}{\vdash \mathtt{A} \to \mathtt{B} \to \mathtt{C} \to \mathtt{B}}\,(\to\mathsf{E})$$

normalises to

$$\dfrac{\dfrac{\dfrac{}{\mathtt{A}, \mathtt{B}, \mathtt{C} \vdash \mathtt{B}}\,(\mathsf{A})}{\mathtt{A}, \mathtt{B} \vdash \mathtt{C} \to \mathtt{B}}\,(\to\mathsf{I})}{\mathtt{A} \vdash \mathtt{B} \to \mathtt{C} \to \mathtt{B}}\,(\to\mathsf{I})}{\not{\mathtt{B}} \not{\to} \not{\mathtt{C}} \not{\to} \not{\mathtt{B}} \vdash \mathtt{A} \to \mathtt{B} \to \mathtt{C} \to \mathtt{B}}\,(\to\mathsf{I})$$

The corresponding reduction is

$$(\lambda\,\mathtt{x}.\ \lambda\,\mathtt{y}.\ \mathtt{x})\ (\lambda\,\mathtt{z}.\ \lambda\,\mathtt{w}.\ \mathtt{z}) \ \to_\beta\ \lambda\,\mathtt{y}.\ \lambda\,\mathtt{z}.\ \lambda\,\mathtt{w}.\ \mathtt{z}$$

Subject reduction and strong normalisation

For simply typed λ-calculus we have the following results.

Theorem (type preservation)**.** If $\Gamma \vdash t : \tau$ and $t \twoheadrightarrow_\beta t'$, then $\Gamma \vdash t' : \tau$.

Theorem (strong normalisation)**.** Every reduction sequence of a well typed λ-term is finite.

They are readily translated into theorems about derivations.

Theorem. Elimination of a detour produces a derivation with the same conclusion.

Theorem. Every derivation can be normalised (to a derivation that does not contain detours).

Canonicity

Definition. A derivation is in *canonical form* exactly when its bottom-most rule is an introduction rule.

Theorem (canonicity)**.** If $\vdash_{\mathrm{NJ}} \varphi$ is derivable, then there is a derivation of $\vdash_{\mathrm{NJ}} \varphi$ in canonical form.

PROOF There is a derivation of $\vdash \varphi$ with no detours; perform induction on this derivation. If the bottom-most rule were an elimination rule, a detour would arise, contradicting the assumption about the derivation.

Exercise. Expand the above proof sketch.

Question. Does the proof above use the principle of indirect proof?

Question. Is it important for a deduction system to have strong normalisation and canonicity?

Question. Why is the context empty in the canonicity statement?

Classical axioms

We obtained the classical deduction system NK by adding to NJ an inference rule, which corresponds to adding a new kinds of term, say

$$\frac{}{\Gamma \vdash \mathtt{LEM}\ \varphi : \varphi \vee \neg\varphi}\ (\mathsf{LEM})$$

We do not know how to reduce `LEM`, however. This breaks canonicity, and the deduction system ceases to be computationally meaningful.

Underivability

Corollary. NJ is consistent, that is, $\nvdash_{\mathrm{NJ}} \bot$.

PROOF If $\vdash_{\mathrm{NJ}} \bot$, then there is a λ-term of type $\bot$ in canonical form. But none of the canonical forms can have type $\bot$.

Corollary (disjunction property)**.** If $\vdash_{\mathrm{NJ}} \varphi \vee \psi$, then either $\vdash_{\mathrm{NJ}} \varphi$ or $\vdash_{\mathrm{NJ}} \psi$.

PROOF A closed λ-term of type $\varphi \vee \psi$ can be reduced to either `left` p where $\vdash p : \varphi$ or `right` q where $\vdash q : \psi$.

Remark. The disjunction property does not hold for NK.

Corollary. $\nvdash_{\mathrm{NJ}} \mathtt{A} \vee \neg\mathtt{A}$.

PROOF If $\vdash_{\mathrm{NJ}} \mathtt{A} \vee \neg\mathtt{A}$, then either $\vdash_{\mathrm{NJ}} \mathtt{A}$ or $\vdash_{\mathrm{NJ}} \neg\mathtt{A}$ by the disjunction property, and thus either $\models \mathtt{A}$ or $\models \neg\mathtt{A}$ by soundness. But neither $\mathtt{A}$ nor $\neg\mathtt{A}$ is a tautology.

Unifying programming and reasoning

The Curry–Howard correspondence suggests that programs and proofs be identified. Both of them are *mental constructions*, which are exactly what intuitionistic mathematics cares about.

- Per Martin-Löf [1984]. Constructive mathematics and computer programming. *Philosophical Transactions of the Royal Society of London*, 312(1522):501–518. https://doi.org/10.1098/rsta.1984.0073.

Martin-Löf Type Theory

Martin-Löf Type Theory is an influential framework in which programs and proofs are treated uniformly. It is simultaneously

- a computationally meaningful higher-order logic system and
- a very expressively typed functional programming language.

There are numerous variations, extensions, and applications of MLTT. The *dependently typed* programming language *Agda* that we will see next is one of its descendants.